# Project - Low Level Design

# On

# Non Profit Quiz /Tutor Bot

Course Name:

Generative AI

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1 | AAYUSH BANWADIKAR | EN22ME304002 |
| 2 | DIVYANSHU MEHTA | EN22EL301024 |
| 3 | ADITYA KUMAR SAHU | EN22ME304005 |
| 4 | VINAY PATIDAR | EN22CS3011085 |
| 5 | YATENDRA VERMA | EN22ME304118 |

*Group Name: 02D10*

*Project Number: GAI-15*

*Industry Mentor Name: Mr. Yash Mittal*

*University Mentor Name: Prof. Divya Kumawat*

*Academic Year: 2026*

# Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Sequence Diagram
   2.6. Key Design Considerations
   2.7. Class diagram
   2.8. API Catalogue

3. References

# 1. Introduction

## 1.1 Scope of the document

The purpose of this document is to describe the design, architecture, and functionality of the Generative AI-based Non-Profit Quiz /Tutor Bot. This document outlines the overall system structure, application workflow, data flow, and technical components used to build the solution.

The system is designed to assist users in learning about the non-profit sector through an interactive quiz-based educational platform powered by Generative Artificial Intelligence and Large Language Models (LLMs). The bot evaluates user responses, provides contextual explanations for incorrect answers, and helps reinforce learning through intelligent feedback.

This document covers the system design, process flow, component architecture, API integrations, and design considerations required to implement the application. It serves as a technical reference for developers, researchers, and stakeholders involved in the development and maintenance of the system.

## 1.2 Intended Audience

This document is intended for individuals involved in the design, development, deployment, and evaluation of the Non-Profit Quiz Tutor Bot system. The primary audience includes:

- Software Developers and Engineers who will implement and maintain the system architecture.
- AI/ML Engineers responsible for integrating Large Language Models and vector databases for intelligent responses.
- Researchers and Students studying Generative AI applications in education and non-profit domains.
- Non-Profit Organizations interested in adopting AI-based learning tools for training and awareness.

The document provides both technical and conceptual information to help these audiences understand how the system operates and how its components interact.

### 1.3 System Overview

The Non-Profit Quiz Tutor Bot is an AI-driven educational platform designed to help users improve their knowledge of non-profit sector concepts through an interactive learning experience.

The system presents users with quiz questions derived from donor communications, fundraising strategies, and non-profit operational concepts. After the user submits an answer, the system evaluates the response using a Large Language Model (LLM). Instead of only indicating whether the answer is correct or incorrect, the AI generates detailed explanations and contextual feedback to help users understand the correct concept.

The system also uses a Vector Database to store knowledge related to non-profit topics, allowing the AI model to retrieve relevant information and provide accurate explanations. The chatbot interface enables users to interact with the system naturally, making the learning experience more engaging.

Key technologies used in the system include:

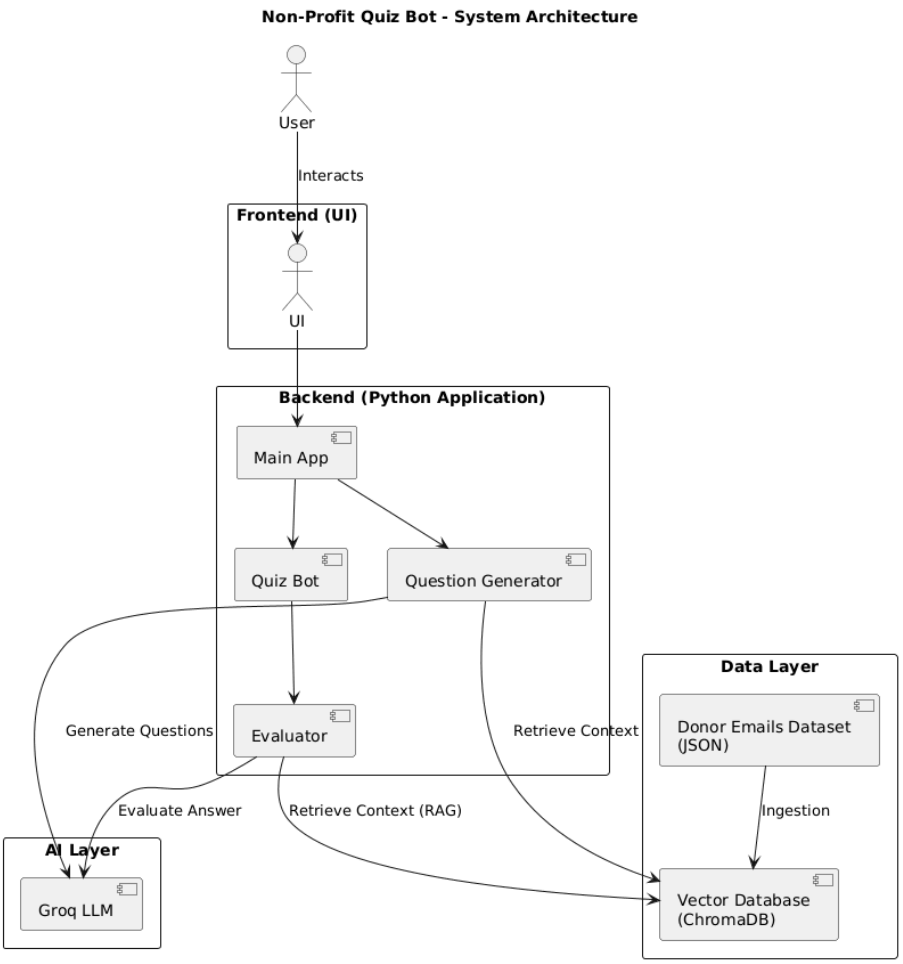
- **Python** for backend development
- **Large Language Models (LLMs)** for intelligent evaluation and explanation
- **Vector Databases (FAISS/ChromaDB)** for semantic search
- **HTML, CSS and JavaScript** for building the chatbot interface
- **API integrations** for connecting the application with AI models

## 2. System Design

### 2.1 Application Design

The Non-Profit Quiz/Tutor Bot is designed as a modular AI-driven system that processes donor email data to generate and evaluate learning assessments. The application begins by loading donor emails from a JSON dataset, which are then split into smaller chunks and converted into embeddings for storage in a vector database (ChromaDB). This enables efficient semantic retrieval of relevant information. The question generation module uses a Large Language Model (LLM) via the Groq API to create context-based quiz questions from the stored data. During the quiz, the user interacts with the system through the quiz bot module, which displays questions and collects responses. The evaluator module then analyzes

the user's answers using a combination of direct matching (for objective questions) and LLM-based evaluation (for descriptive answers). It also retrieves relevant email context using a Retrieval-Augmented Generation (RAG) approach to generate accurate and meaningful explanations. The system is controlled by a main controller that manages data ingestion, quiz flow, and evaluation, thereby creating a personalized learning loop that helps users improve their understanding of nonprofit communication practices.




The Non-Profit Quiz Tutor Bot follows a layered application architecture consisting of four main layers: Frontend (User Interface), Backend (Python Application), AI Layer, and Data Layer. Each layer performs a specific function and communicates with other components to deliver an interactive AI-based learning experience. The architecture ensures modularity, scalability, and efficient data processing.

## 1. User Interface (Frontend Layer)

The Frontend Layer acts as the interaction point between the user and the system. Users access the application through a chatbot-style interface where they can answer quiz questions and receive explanations.

The interface is designed to be simple and interactive so that users can easily engage with the learning system. It sends user responses to the backend for processing and displays the feedback generated by the AI system. Technologies such as Streamlit or web-based UI frameworks can be used to build this layer.

The main responsibilities of the frontend include:

- Accepting user responses to quiz questions
- Displaying quiz questions generated by the system
- Showing evaluation results and AI-generated explanations
- Managing user interaction with the chatbot interface

## 2. Backend Layer (Python Application)

The Backend Layer is the core processing unit of the application. It manages the logic required for generating quiz questions, evaluating answers, and retrieving contextual knowledge from the database.

The backend consists of several modules:

- **Main Application Module:** The main application module controls the overall workflow of the system. It manages communication between the frontend, quiz engine, evaluation system, AI model, and database.

- **Quiz Bot Module:** The Quiz Bot module manages quiz interactions. It sends quiz questions to the user, collects answers, and forwards them for evaluation.

- **Question Generator:** The evaluator analyzes the user's answers and determines whether they are correct or incorrect. It sends the response to the AI model for deeper analysis and explanation generation.

## 3. AI Layer -

The AI Layer is responsible for intelligent reasoning and explanation generation. This layer integrates a Large Language Model (LLM) **Groq API - LLaMA 3.1** to perform natural language understanding and generate contextual responses.

The AI model performs the following functions:

- Evaluates the correctness of user answers
- Generates explanations for incorrect answers
- Provides contextual feedback to enhance learning
- Uses retrieved knowledge to ensure accurate responses

This layer uses a Retrieval-Augmented Generation (RAG) approach, where the AI model retrieves relevant information from the vector database before generating responses. This improves accuracy and reduces hallucinations.

## 4. Data Layer

The Data Layer manages the storage and retrieval of knowledge used by the system. It contains structured and processed information related to non-profit sector topics.

The main components of this layer include:

- **Donor Emails Dataset (JSON):** The dataset contains donor communication examples and non-profit related content stored in JSON format. This dataset is used to train and support the quiz generation process.
- **Vector Database (ChromaDB):** The dataset is converted into embeddings and stored in a vector database. This allows the system to perform semantic search and retrieve relevant context when evaluating answers or generating questions.

The vector database supports the **Retrieval-Augmented Generation (RAG)** mechanism, enabling the AI model to access accurate contextual information before generating explanations.

### 5. System Workflow

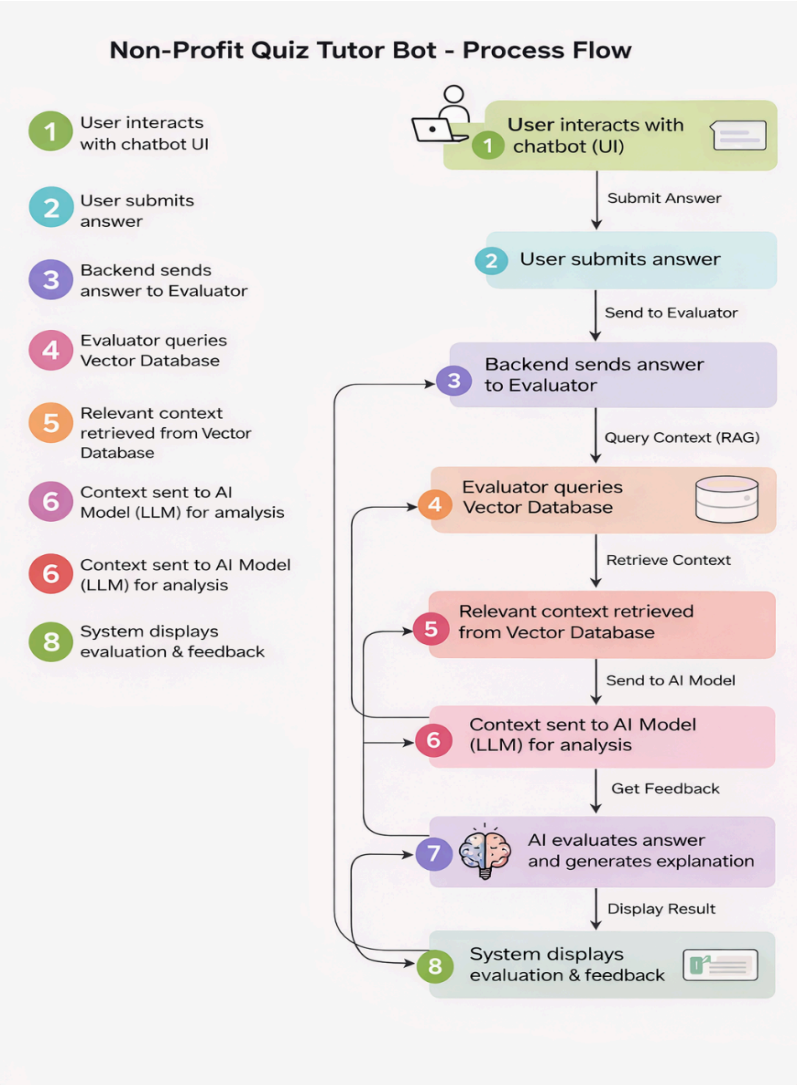
The interaction between different components follows a structured workflow:

1. The user interacts with the **frontend interface** and submits an answer.
2. The frontend sends the response to the **backend main application**.

3. The **Quiz Bot module** manages the quiz session and forwards the answer to the **Evaluator module**.
4. The evaluator retrieves relevant context from the **Vector Database (ChromaDB)** using semantic search.
5. The retrieved information is sent to the **Groq LLM** in the AI layer.
6. The AI model evaluates the answer and generates an explanation.
7. The result is returned to the **backend** and displayed to the **user interface**.

This architecture enables the system to deliver an **interactive AI-powered tutoring experience**, helping users understand non-profit concepts through personalized feedback and explanations.

## 2.2 Process Flow



## 2.3 Information Flow

| | |
|---|---|
| User Input | The user enters responses through the chatbot interface. |
| Input Processing | The application processes the input and sends it to the evaluation module. |
| Knowledge Retrieval | The system queries the vector database to retrieve relevant information. |
| AI Analysis | The LLM analyzes the response and generates an explanation. |
| Output Generation | The processed output is sent back to the chatbot interface. |
| User Feedback | The user receives the result along with educational feedback. |

## 2.4 Components Design

The system consists of several components that work together to deliver the final functionality.

**1. Chatbot Interface**
Provides a conversational interface where users answer quiz questions and receive feedback.

**2. Quiz Engine**
Responsible for generating quiz questions and managing question sequences.

**3. Response Evaluation Module**
Uses the Large Language Model to assess the correctness of answers.

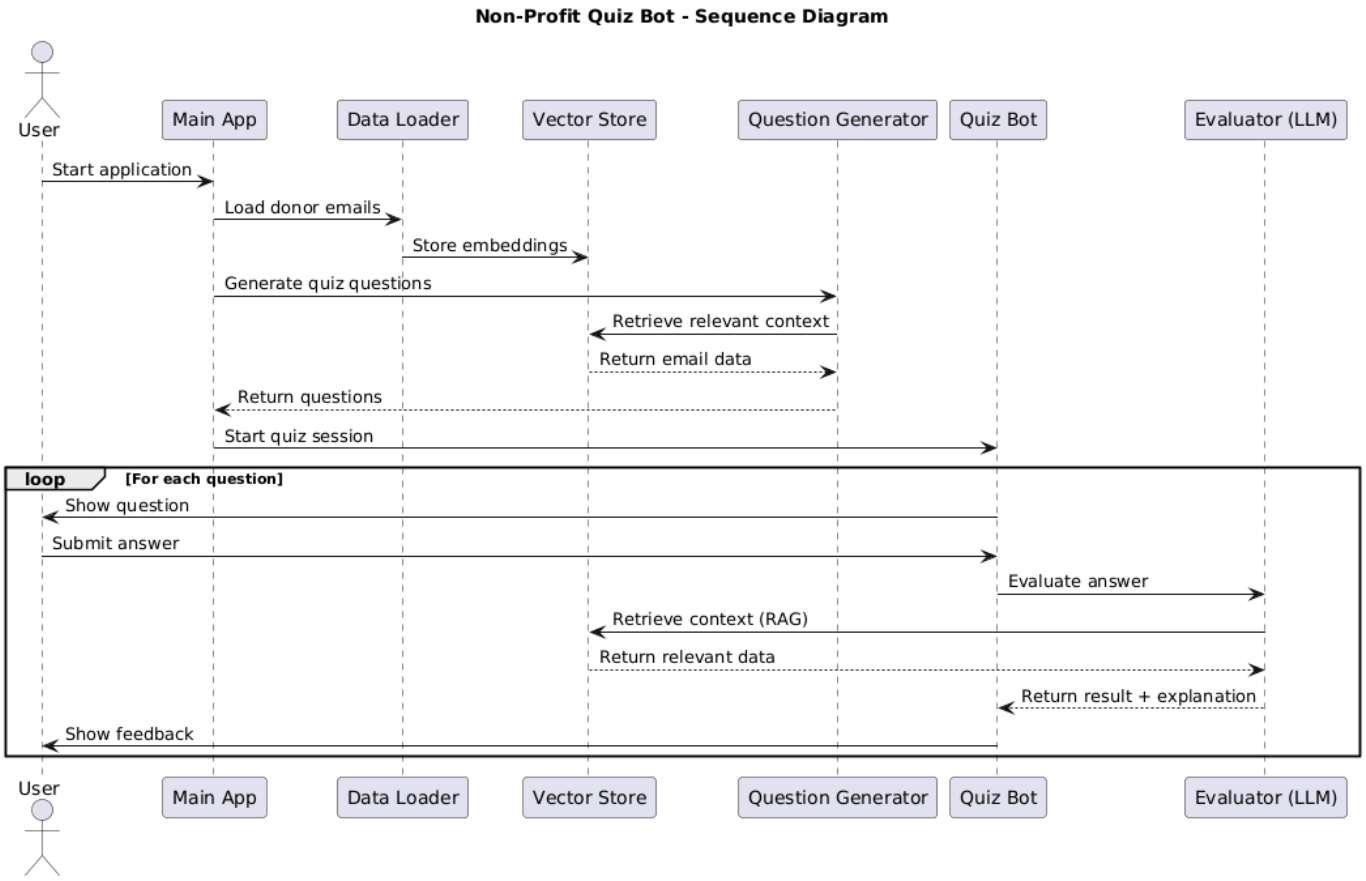
**4. Knowledge Retrieval Module**
Retrieves relevant information from the vector database using semantic search.

**5. Explanation Generator**
Generates detailed explanations to help users understand mistakes.

**6. Data Storage Module**

Stores quiz questions, user responses, and system logs for analysis.Each component performs a specific function while interacting with other modules through defined interfaces.

## 2.5 Sequence Diagram




1. The user starts the application, and the request is sent to the Main Application.
2. The Main Application loads the donor email dataset using the Data Loader module.
3. The Data Loader converts the dataset into embeddings and stores them in the Vector Store (Vector Database).
4. The Question Generator retrieves relevant information from the Vector Store and generates quiz questions.
5. The Quiz Bot starts the quiz session and displays questions to the user.
6. The User submits an answer for the displayed question.
7. The Evaluator (LLM) analyzes the answer and retrieves relevant context from the Vector Store using RAG.
8. The AI model evaluates the answer and generates a result with explanation.
9. The Quiz Bot displays the feedback to the user.
10. This process repeats for each question until the quiz session is completed.

## 2.6 Key Design Considerations

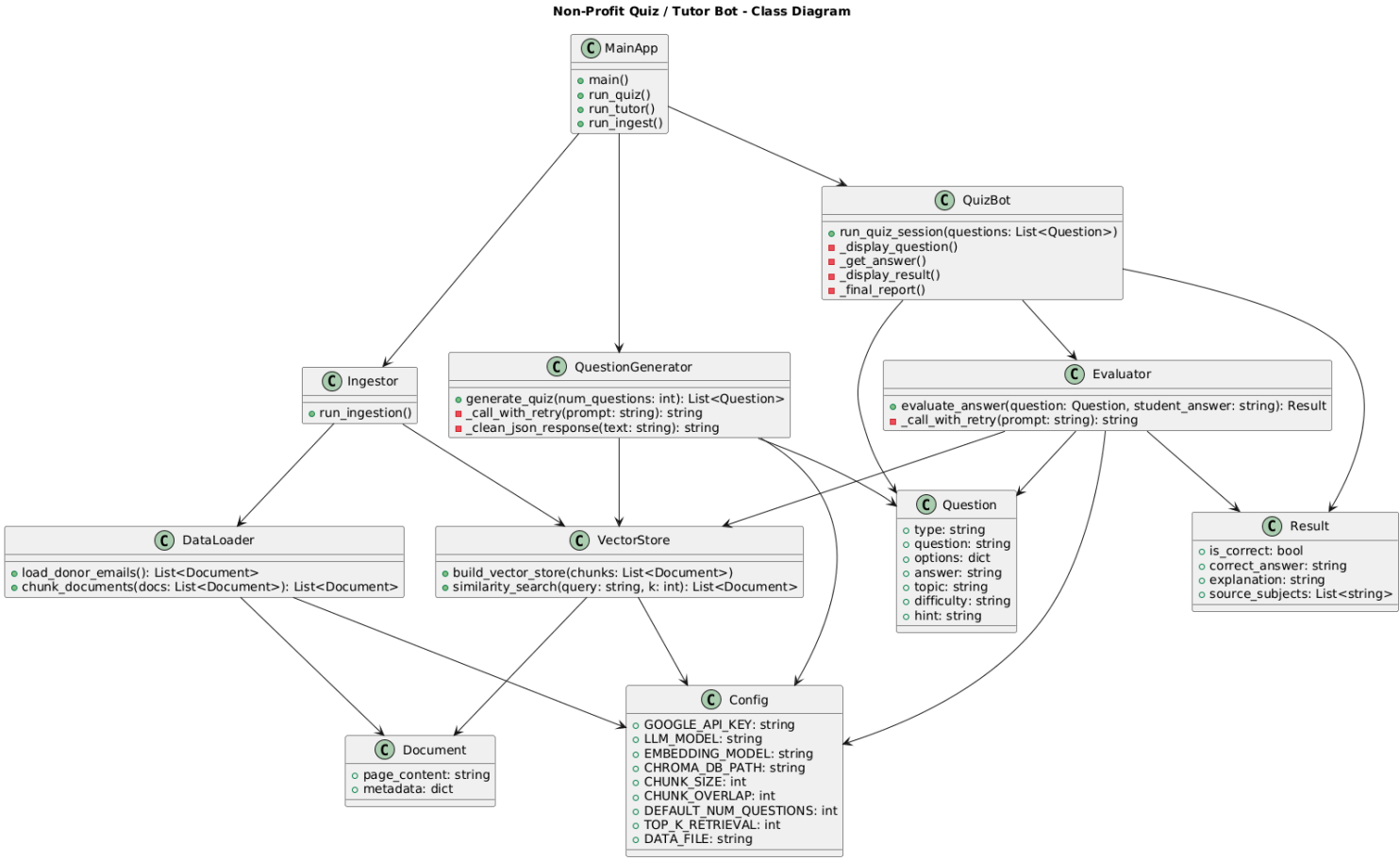
- **Scalability**
  The system architecture allows easy expansion to support more users and larger knowledge bases.
- **Accuracy of AI Responses**
  The use of a vector database ensures that the AI retrieves reliable and relevant knowledge.
- **User Experience**
  The chatbot interface is designed to be simple, interactive, and user-friendly.
- **Modularity**
  Different components are separated so that updates can be implemented without affecting the entire system.
- **Security and Privacy**
  User data and responses are handled securely to ensure privacy and data protection.
- **Performance Efficiency**
  Efficient retrieval mechanisms and optimized AI queries ensure fast response times.

## 2.7 Class Diagram

1. **MainApp Class**
   - This is the **main controller of the system**.
   - It starts the application and manages the overall workflow.
   - It runs the **quiz, tutor functions, and data ingestion process**.

2. **Ingestor Class**
   - Responsible for **loading and processing the dataset**.
   - It prepares the donor email data so it can be used by the system.

3. **DataLoader Class**
   - Loads the **donor email dataset** from files.
   - Breaks large documents into **smaller chunks** so they can be processed easily.




4. **VectorStore Class**
   - Stores the processed data in a vector database (ChromaDB).
   - Performs similarity search to retrieve relevant information when needed.

5. **QuestionGenerator Class**
   - Generates quiz questions from the stored knowledge.
   - Uses prompts and AI models to create questions related to non-profit topics.

6. **QuizBot Class**
   - Manages the quiz session with the user.
   - Displays questions, collects answers, and shows results.

7. **Evaluator Class**
   - Evaluates the user's answer using a Large Language Model (LLM).
   - Determines if the answer is correct and generates explanations.

8. **Question Class**
   - Represents the structure of a quiz question.
   - Stores details like question text, options, answer, topic, difficulty, and hint.

9. **Result Class**
   - Stores the evaluation result of the user's answer.
   - Includes whether the answer is correct, the correct answer, and the explanation.

10. **Document Class**
    - Represents the content of donor email data.
    - Contains the document text and related metadata.

## 2.8 API Catalogue

| API Name | Module | Input | Output | Purpose |
|---|---|---|---|---|
| load_donor_emails() | Data Loader | JSON file path | List of Documents | Loads donor email dataset |
| chunk_documents() | Data Loader | List of Documents | Chunked Documents | Splits emails into smaller parts |
| build_vector_store() | Vector Store | Document chunks | Vector DB created | Stores embeddings in ChromaDB |
| similarity_search() | Vector Store | Query, k | Relevant Documents | Retrieves related email context |

| | | | | |
|---|---|---|---|---|
| generate_quiz() | Question Generator | Number of questions | List of Questions | Generates quiz using LLM |
| evaluate_answer () | Evaluator | Question, User Answer | Result (correct/incorrect + explanation) | Evaluates answers using LLM |
| run_ingestion() | Ingestion | None | Vector DB ready | Executes full data pipeline |
| run_quiz_session() | Quiz Bot | List of Questions | Score + Feedback | Runs quiz interaction |
| main() | Main Controller | CLI arguments | System execution | Controls overall application flow |

## 3. References

1. **Groq API Documentation**
   Groq Inc.
   Available at: https://console.groq.com/docs
   (Accessed: March 2026)

2. **LLaMA 3.1 Model (Meta AI)**
   LLaMA 3.1 – Meta AI Research
   Available at: https://ai.meta.com/llama/
   (Accessed: March 2026)

3. **LangChain Documentation**
LangChain
Available at: https://python.langchain.com
(Accessed: March 2026)

4. **ChromaDB Documentation**
Chroma
Available at: https://docs.trychroma.com
(Accessed: March 2026)

5. **Retrieval-Augmented Generation (RAG)**
Retrieval-Augmented Generation – Lewis et al., Facebook AI Research
Available at: https://arxiv.org/abs/2005.11401

6. **Sentence Transformers (Embeddings)**
Reimers, N., & Gurevych, I. (2019)
"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks"
Available at: https://www.sbert.net

7. **Python Documentation**
Python Software Foundation
Available at: https://docs.python.org/3/